Topological Ring Detection for Market Manipulation

Optimization report — gradient descent/ascent applied to wash-trading ring detection

Gopreeth · 25 October 2026

Abstract

Wash trading inflates market volume by moving assets around closed rings of colluding accounts. Each trade looks normal; the ring does not. This report models each time window of transactions as a weighted directed graph and measures how strongly each loop persists using persistent homology on a directed flag complex. In that complex, circular flows survive and one-way flows are filled in.

Three parts of the detector are posed as optimization problems and solved with gradient methods:

- a convex, regularized logistic classifier trained by gradient descent;
- the weights that combine three edge-level suspicion scores, learned by projected gradient descent on the probability simplex using a smooth (log-sum-exp) ring birth;
- an alarm threshold that tracks drifting tactics by online gradient ascent on a smoothed reward with delayed labels.

On synthetic markets with planted wash rings and realistic legitimate loops (market makers, arbitrage bots), learning the weights raised the test average precision of the ring-birth detector from 0.568 to 0.9997. The learned weights drop reciprocity entirely, a choice the KKT conditions confirm. A step size matched to the local curvature made gradient descent converge in 322 iterations where the textbook bound had not converged after 3,000. Under tactic drift, online gradient ascent kept 97% of the oracle reward, against 24% for a fixed threshold.

1. Problem Description

The problem: find groups of colluding accounts that trade with each other in a circle to fake market activity (wash trading), score how suspicious each group is, and keep the detection threshold accurate as manipulators change tactics.

1.1 What wash trading is

In a wash trade, the same party (or a coordinated group) is on both sides of a trade. No asset really changes hands economically, but the trade is recorded as market volume. The purpose is to mislead other traders: fake volume makes an asset look popular and liquid, can push its price up, and can earn trading rewards on platforms that pay for volume. Wash trading is illegal in regulated securities markets and is widespread in crypto-asset and NFT markets, where accounts are cheap to create and pseudonymous.

A single wash trade looks normal. The give-away is the **pattern of connections**:

- **Two-party ping-pong:** account A sells to B, then B sells back to A, again and again (A ⇄ B).
- **Multi-party ring:** value moves A → B → C → A and returns to where it started, so every account ends the period with roughly zero net position.
- **Regular cadence:** colluding accounts are usually bots, so the trades repeat on a near-fixed schedule with near-equal amounts.

1.2 Why existing approaches fall short

1. **Density-based methods miss sparse rings.** Most graph fraud detectors (community detection, centrality, dense-subgraph mining) look for tightly connected clusters. A wash ring can be sparse: each account trades with only two others, yet the group forms a closed loop.
2. **Cycle detection is binary.** Finding a cycle says that a loop exists, but not how strong, stable or suspicious it is. Legitimate actors (arbitrage bots, market makers) also create loops.
3. **Fixed thresholds decay.** Manipulators adapt: they add timing noise, vary amounts, or route through more accounts. A threshold tuned once on past data loses precision and recall as tactics drift.

1.3 Approach and the role of optimization

The project models transactions in each time window as a **directed weighted graph** and uses **persistent homology on a directed flag complex** to measure how strongly each loop persists (Section 2.3). Three parts of the pipeline are posed as optimization problems solved by gradient methods:

Part	Optimization problem	Method
Ring classifier	Minimise regularised cross-entropy loss (convex)	Batch gradient descent
Edge-weight design	Learn how to combine three suspicion scores, weights constrained to a probability simplex (non-convex, constrained)	Projected gradient descent
Adaptive threshold	Maximise a smoothed reward (catches − cost × false alarms) that drifts over time	Online gradient ascent with delayed labels

1.4 Scope

The target is **closed-cycle wash trading** in settled transaction data. Spoofing (orders placed and cancelled without executing) is excluded because it is invisible in settled transactions. All experiments in this report use a synthetic transaction generator with planted wash rings and realistic legitimate loops, because it provides exact ground truth; the same pipeline is designed to run on the IBM AML benchmark and Ethereum NFT trades in later phases.

2. Mathematical Formulation

The problem is formulated in four layers: a weighted directed graph per time window, a topological filtration on it, and three optimization problems solved by gradient descent or ascent: a convex classifier (2.5), a constrained non-convex weight-learning problem (2.6) and an online threshold problem (2.7).

2.1 Transaction graph

A transaction is a tuple (t_i, u_i, v_i, a_i): time, sender account, receiver account, amount. For window length L and stride S, window k covers the interval below. Its transactions form a directed

graph G_k = (V_k, E_k) with one edge per ordered account pair.

$$W_k = [\, kS,\ kS + L\,), \qquad (a,b) \in E_k \iff \exists\, i :\ t_i \in W_k,\ u_i = a,\ v_i = b$$

For each edge e = (a, b) we aggregate its volume, its transaction count, and the gaps between consecutive transaction times:

$$V_{ab} = \sum_{i:\, u_i = a,\, v_i = b,\, t_i \in W_k} a_i, \qquad n_{ab} = \left|\{i : u_i = a,\, v_i = b,\, t_i \in W_k\}\right|, \qquad g_j = t_{(j+1)} - t_{(j)}$$

2.2 Edge suspicion scores and the filtration weight

Three scores in [0, 1] describe wash-trading behaviour on each edge (higher = more suspicious).

Pair reciprocity: equal volume both ways between the same two accounts.

$$s_{\text{rec}}(a,b) = 1 - \frac{|V_{ab} - V_{ba}|}{V_{ab} + V_{ba}}$$

Node flow balance: in a ring every member ends the window close to net zero. With in-volume and out-volume of account x:

$$\beta(x) = 1 - \frac{|\text{in}(x) - \text{out}(x)|}{\text{in}(x) + \text{out}(x)}, \qquad s_{\text{bal}}(a,b) = \min\{\beta(a),\, \beta(b)\}$$

Timing regularity: bots trade on a fixed cadence, so the coefficient of variation (CV) of the gaps is small. Pairs with fewer than 3 transactions get the window median, so they are biased neither way.

$$\text{CV}_{ab} = \frac{\text{std}(g)}{\text{mean}(g)}, \qquad s_{\text{reg}}(a,b) = \frac{1}{1 + \text{CV}_{ab}}$$

Rank normalization. Each score is replaced by its rank among the m edges of the window, with ties taking the lowest rank, so that the large block of edges with s_rec = 0 maps to 0:

$$z_{e,j} = \frac{\text{rank}_{\min}(s_j(e)) - 1}{m - 1} \in [0,1], \qquad z_e = (z_{e,\text{rec}},\, z_{e,\text{bal}},\, z_{e,\text{reg}})^\top$$

Filtration weight. The scores are combined with a weight vector α that lies on the probability simplex Δ. A lower edge weight means a more suspicious edge.

$$w_e(\alpha) = 1 - \alpha^\top z_e, \qquad \alpha \in \Delta = \{\alpha \in \mathbb{R}^3 : \alpha_j \geq 0,\ \textstyle\sum_j \alpha_j = 1\}$$

Since z_e is in [0,1]^3 and α is in Δ, w_e is always in [0, 1]. The baseline uses equal weights α = (1/3, 1/3, 1/3).

Proposition 1 (the 1/3 floor). In a ring of three or more accounts with no reciprocal pairs, z_rec = 0 on every edge, so for any α:

$$w_e(\alpha) = 1 - \alpha_{\text{bal}} z_{e,\text{bal}} - \alpha_{\text{reg}} z_{e,\text{reg}} \;\geq\; 1 - \alpha_{\text{bal}} - \alpha_{\text{reg}} = \alpha_{\text{rec}}$$

With equal weights, no multi-party ring can score below 1/3, however perfect its balance and timing. This is observed in the experiments (Section 4) and is the motivation for *learning* α in Section 2.6 instead of fixing it.

2.3 Directed flag complex and persistent homology

Directed flag complex. Given G and weights w, the complex has a 0-simplex for each account, a 1-simplex for each directed edge (a reciprocal pair gives two), and a 2-simplex for each ordered triple whose edges form a one-way flow:

$$\sigma = (v_0, v_1, v_2) \text{ is a 2-simplex } \iff (v_0, v_1),\ (v_0, v_2),\ (v_1, v_2) \in E$$

Filtration. Each simplex enters at the largest weight of its edges (vertices at 0). The sublevel complexes are nested, which is what makes persistence well defined:

$$f(\sigma) = \max_{e \in \sigma} w_e, \qquad K_t = \{\sigma : f(\sigma) \leq t\}, \qquad K_s \subseteq K_t \text{ for } s \leq t$$

Homology over Z/2. With boundary maps between chain groups (the boundary of a boundary is zero), the first homology counts independent loops that are not filled in:

$$\partial_1(a, b) = a + b, \qquad \partial_2(v_0, v_1, v_2) = (v_1, v_2) + (v_0, v_2) + (v_0, v_1), \qquad \partial_1 \circ \partial_2 = 0$$

$$H_1(K_t) = \ker \partial_1 \,/\, \operatorname{im} \partial_2$$

A loop class that first appears at t = b and becomes a boundary at t = d gives a persistence pair (b, d); d = ∞ if it is never filled. Long bars (large d − b) born early (small b) are the signature of a strong, suspicious loop. The pairs are computed by reducing the boundary matrix: column additions over Z/2 until every column has a distinct lowest non-zero entry, each such entry pairing a birth simplex with a death simplex.

Proposition 2 (cyclic triangles are never filled). A 2-simplex needs a vertex v_0 with edges to both other vertices. In a cyclic triangle A → B → C → A every vertex has exactly one outgoing edge inside the triangle, so no ordering exists. Its loop (A,B) + (B,C) + (C,A) has zero boundary but is not the boundary of anything, so it persists: bar (b, ∞).

Proposition 3 (transitive triangles die at once). For A → B, B → C, A → C, the ordering (A, B, C) is a 2-simplex entering at the same value as its last edge, so the loop has zero persistence. One-way flow is not a ring.

Proposition 4 (filling through a fourth account). Add account D with D → A, D → B, D → C (weight 0.6) to a cyclic triangle (weight 0.1). Then (D,A,B), (D,B,C) and (D,C,A) are 2-simplices, and over Z/2 every D-edge appears twice and cancels:

$$\partial_2\big[(D,A,B)+(D,B,C)+(D,C,A)\big]=(A,B)+(B,C)+(C,A)$$

The ring is born at 0.1 and dies at 0.6: bar (0.1, 0.6). This is where topology adds information that plain cycle enumeration does not: neighbouring trades can *explain away* a loop.

Ring birth. For a candidate ring R (a simple directed cycle of length 2–6 found by enumeration inside strongly connected components), its loop exists from the moment its last edge enters:

$$b_R(\alpha)=\max_{e\in R} w_e(\alpha)$$

2.4 Learning tasks and data

Each candidate ring R has a label y_R = 1 if it is a planted wash ring and 0 otherwise (background loops, arbitrage loops, market-maker loops). Positives are rare, so the loss gives each class equal total weight:

$$\omega_R=\frac{N}{2N_{y_R}},\qquad N_1=\#\{R:y_R=1\},\quad N_0=\#\{R:y_R=0\}$$

2.5 Problem A: ring classifier by gradient descent (convex)

Model. For a standardized feature vector x_R (topological and behavioural features of the ring, plus a bias entry 1):

$$p_R=\sigma(\theta^\top x_R),\qquad \sigma(u)=\frac{1}{1+e^{-u}},\qquad \sigma'(u)=\sigma(u)\,(1-\sigma(u))$$

Objective (weighted, L2-regularized cross-entropy; the bias is not penalized):

$$\min_{\theta\in\mathbb{R}^d}\ J(\theta)=-\frac{1}{N}\sum_R \omega_R\big[\,y_R\log p_R+(1-y_R)\log(1-p_R)\,\big]+\frac{\lambda}{2}\|\tilde\theta\|^2$$

Gradient. With u = θᵀx, the per-sample derivative of the loss with respect to u simplifies to p − y (chain rule with σ′ = σ(1 − σ)):

$$\frac{\partial \ell_R}{\partial u}=-y_R(1-p_R)+(1-y_R)\,p_R=p_R-y_R$$

$$\nabla J(\theta)=\frac{1}{N}X^\top\Omega\,(p-y)+\lambda\tilde\theta,\qquad \Omega=\mathrm{diag}(\omega_R)$$

Convexity. The Hessian is positive semi-definite, since p(1 − p) ≥ 0 and the weights are positive; so every stationary point is a global minimum:

$$\nabla^2 J(\theta)=\frac{1}{N}X^\top\Omega S X+\lambda\tilde I\succeq 0,\qquad S=\mathrm{diag}\,\big(p_R(1-p_R)\big)$$

Smoothness constant. Because p(1 − p) ≤ 1/4, the gradient is L-Lipschitz with

$$L = \frac{1}{4N}\,\lambda_{\max}(X^\top \Omega X) + \lambda$$

Gradient descent and its guarantee. The update and the descent lemma (from the L-smoothness upper bound):

$$\theta^{t+1} = \theta^t - \eta\,\nabla J(\theta^t)$$

$$J(\theta^{t+1}) \le J(\theta^t) - \eta\left(1 - \frac{L\eta}{2}\right)\|\nabla J(\theta^t)\|^2$$

So any step 0 < η < 2/L decreases J, η = 1/L gives the largest guaranteed decrease, and for convex J:

$$J(\theta^t) - J^* \le \frac{\|\theta^0 - \theta^*\|^2}{2\eta t}, \qquad \eta \le \frac{1}{L}$$

If the penalty makes J μ-strongly convex, convergence is linear (geometric):

$$\|\theta^t - \theta^*\|^2 \le (1 - \eta\mu)^t\,\|\theta^0 - \theta^*\|^2$$

Steps above 2/L can overshoot and diverge; Section 4.2 shows all three regimes.

2.6 Problem B: learning the edge-weight combination (constrained, non-convex)

Instead of fixing α = (1/3, 1/3, 1/3), α is learned so that wash rings get low birth values and legitimate loops get high ones.

Smooth ring birth. The max in b_R is not differentiable, so it is replaced by the log-sum-exp (soft maximum) with sharpness β:

$$b_R^\beta(\alpha) = \frac{1}{\beta}\log\sum_{e\in R}\exp\big(\beta\, w_e(\alpha)\big), \qquad \max_{e\in R} w_e \;\le\; b_R^\beta \;\le\; \max_{e\in R} w_e + \frac{\log|R|}{\beta}$$

Its gradient is a softmax-weighted average over the ring's edges. The weights concentrate on the ring's highest-weight edge, its weakest link:

$$\pi_{R,e} = \frac{e^{\beta w_e}}{\sum_{e'\in R} e^{\beta w_{e'}}}, \qquad \frac{\partial b_R^\beta}{\partial \alpha} = \sum_{e\in R}\pi_{R,e}\frac{\partial w_e}{\partial \alpha} = -\sum_{e\in R}\pi_{R,e}\, z_e$$

Model and objective. A ring is called suspicious when its smooth birth is low; c is an offset and k > 0 a slope:

$$p_R = \sigma\big(c - k\, b_R^\beta(\alpha)\big)$$

$$\min_{\alpha,\,c,\,k}\; J(\alpha,c,k) = -\frac{1}{N}\sum_{R}\omega_R\big[\,y_R\log p_R + (1-y_R)\log(1-p_R)\,\big] \quad \text{s.t.} \quad \alpha \geq 0,\;\; \mathbf{1}^\top\alpha = 1$$

Gradients. With residual r_R = ω_R (p_R − y_R) / N, the chain rule gives:

$$\frac{\partial J}{\partial c} = \sum_R r_R, \qquad \frac{\partial J}{\partial k} = -\sum_R r_R\, b_R^{\beta}, \qquad \nabla_\alpha J = k\sum_R r_R \sum_{e\in R}\pi_{R,e}\, z_e$$

Reading the α-gradient. For a missed wash ring (y = 1, p < 1, so r < 0), the descent step −η∇J adds a positive multiple of the weakest-link score vector z_e to α. The update therefore puts more weight on whichever scores flag that ring's weakest edge. For a false alarm (y = 0, r > 0) it does the opposite.

Non-convexity. J composes a sigmoid with a log-sum-exp of affine functions, so it is not convex in (α, c, k) jointly. Gradient methods converge to a stationary point, not necessarily the global minimum; random restarts check sensitivity.

Projected gradient descent. The c and k steps are ordinary gradient steps; the α step is projected back onto the simplex:

$$\alpha^{t+1} = \Pi_\Delta(\alpha^t - \eta\,\nabla_\alpha J)\,, \qquad \Pi_\Delta(u) = \arg\min_{\alpha\in\Delta}\tfrac{1}{2}\|\alpha - u\|^2$$

Projection via KKT conditions. The Lagrangian of the projection problem, with multiplier ν for the equality and μ ≥ 0 for the bounds, is:

$$\mathcal{L}(\alpha,\nu,\mu) = \tfrac{1}{2}\|\alpha-u\|^2 + \nu\,(\mathbf{1}^\top\alpha - 1) - \mu^\top\alpha$$

The KKT conditions are:

$$\alpha_j - u_j + \nu - \mu_j = 0, \qquad \mu_j \geq 0, \qquad \alpha_j \geq 0, \qquad \mu_j\alpha_j = 0, \qquad \textstyle\sum_j \alpha_j = 1$$

If α_j > 0 then μ_j = 0 and α_j = u_j − ν; if α_j = 0 then u_j − ν = −μ_j ≤ 0. Both cases give:

$$\alpha_j = \max(u_j - \nu,\,0), \qquad \nu \text{ chosen so that } \textstyle\sum_j \max(u_j-\nu,0) = 1$$

The left side of the last equation decreases continuously in ν, so ν is unique. Sort u in decreasing order, u_(1) ≥ u_(2) ≥ u_(3), and take the largest ρ for which the condition below holds; then:

$$\rho = \max\Big\{j : u_{(j)} - \tfrac{1}{j}\big(\textstyle\sum_{i\leq j} u_{(i)} - 1\big) > 0\Big\}, \qquad \nu = \tfrac{1}{\rho}\Big(\textstyle\sum_{i\leq\rho} u_{(i)} - 1\Big)$$

Worked example. u = (0.5, 0.8, −0.1). Sorted: 0.8, 0.5, −0.1. For j = 1: 0.8 − (0.8 − 1) = 1.0 > 0. For j = 2: 0.5 − (1.3 − 1)/2 = 0.35 > 0. For j = 3: −0.1 − (1.2 − 1)/3 ≈ −0.167 < 0. So ρ = 2 and ν = 0.15, giving Π(u) = (0.35, 0.65, 0), which sums to 1.

Stationarity on the simplex. A point α* is stationary for PGD exactly when it satisfies the KKT conditions of Problem B:

$$\alpha^* = \Pi_\Delta(\alpha^* - \eta \nabla_\alpha J(\alpha^*)) \iff \begin{cases} \partial_j J(\alpha^*) = -\nu & \text{if } \alpha_j^* > 0 \\ \partial_j J(\alpha^*) \geq -\nu & \text{if } \alpha_j^* = 0 \end{cases}$$

In words, at the optimum every score in use has the same marginal effect on the loss, and a score that was dropped (weight 0) would not reduce the loss if added back.

2.7 Problem C: adaptive threshold by online gradient ascent

In window t each candidate ring i has a score s_i = b_i (its birth under the learned α) and is flagged if s_i ≤ τ. With a cost λ per false alarm, the reward is:

$$R_t(\tau) = \sum_i g_i \, \mathbf{1}[s_i \leq \tau], \qquad g_i = y_i - \lambda(1 - y_i) \qquad (= \mathrm{TP} - \lambda \, \mathrm{FP})$$

R_t is a step function of τ, so its gradient is zero almost everywhere. The indicator is smoothed with temperature T:

$$\tilde{R}_t(\tau) = \sum_i g_i \, \sigma\Big(\frac{\tau - s_i}{T}\Big), \qquad \frac{d\tilde{R}_t}{d\tau} = \frac{1}{T} \sum_i g_i \, \sigma_i(1 - \sigma_i)$$

Only rings with scores within a few T of the threshold contribute: positives push τ up (catch more) and negatives push it down, weighted by λ.

Optimality condition. As T → 0 the factor σ(1 − σ)/T tends to a delta function, and the stationary condition becomes a density balance between positives (f_+) and negatives (f_−):

$$n_+ f_+(\tau^*) = \lambda \, n_- f_-(\tau^*), \qquad \text{a maximum if } n_+ f_+'(\tau^*) < \lambda \, n_- f_-'(\tau^*)$$

The best threshold is where one more flagged ring is equally likely to be a catch or λ times as likely to be a false alarm. This is the Bayes decision rule written as a gradient condition.

Online gradient ascent with delayed labels. Labels arrive d windows late, so the step at time t uses the latest labelled window and is clipped to [0, 1]; the gradient is normalized by the number of candidates n:

$$\tau_{t+1} = \Pi_{[0,1]}\left(\tau_t + \eta \, \frac{1}{n_{t-d}} \frac{d\tilde{R}_{t-d}}{d\tau}(\tau_t)\right)$$

As tactics drift, the score distributions move and τ* moves with them. Gradient ascent tracks it because every window gives a fresh gradient. This is compared with three alternatives: a fixed threshold chosen before the drift, a threshold re-tuned by grid search every M windows on the same delayed labels, and a hindsight oracle that picks the best τ for each window.

2.8 Evaluation metrics

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}, \qquad F_1 = \frac{2\,PR}{P+R}, \qquad \text{AP} = \sum_n (R_n - R_{n-1})\,P_n$$

Average precision (AP) summarises the precision–recall curve and is preferred to ROC-AUC because positives are rare. Results are reported on windows not used for training (different random seeds), with mean ± standard deviation over seeds.

3. Code Implementation

The pipeline is a Python package, `ringtda`, written in NumPy, pandas, NetworkX, scikit-learn and pyflagser. Every optimization step is written out by hand, so each line maps to an equation in Section 2. It has 29 automated tests, including finite-difference checks of every gradient.

3.1 Structure and pipeline

Module	Role	Report section
`data/synthetic.py`	Transaction generator: background traffic, planted wash rings, arbitrage loops, market makers, drift and evasion settings	1.4, 4.1
`graph/windows.py`, `graph/weights.py`	Windows, per-pair aggregation, scores s_rec, s_bal, s_reg, rank normalization, weight w_e	2.1, 2.2
`topology/candidates.py`	Sparsify, strongly connected components, simple directed cycles of length 2–6	2.3
`topology/persistence.py`	Weighted directed-flag persistence (pyflagser, plus a pure-Python boundary-matrix reduction used to cross-check it)	2.3
`features.py`	Candidate-ring table: labels, behavioural features D, topological features T	2.4
`models/gd.py`	Problem A (logistic regression by gradient descent) and Problem B (α by projected gradient descent)	2.5, 2.6
`threshold/ascent.py`	Problem C (online gradient ascent on the smoothed reward) and the comparison policies	2.7

Module	Role	Report section
`experiments/gd_report.py`	Runs all experiments, writes the figures and `results/gd_metrics.json`	4

The pipeline for one window:

1. Group transactions by ordered account pair; compute the three scores and rank-normalize them (z_e).
2. Build the directed graph with weights w_e(α); drop edges above a loose cutoff; enumerate candidate rings inside strongly connected components.
3. For each candidate: compute ring birth, behavioural features, and directed-flag persistence of its one-hop neighbourhood.
4. Fit the classifier (Problem A) and the score weights α (Problem B) on training windows; evaluate on separate test windows.
5. Run the threshold policies (Problem C) on a 40-window sequence with drifting tactics.

3.2 Edge scores (Section 2.2)

```
def compute_scores(pairs):
    p = pairs.copy()
    # pair reciprocity: 1 - |V_ab - V_ba| / (V_ab + V_ba)
    rev = p[["src", "dst", "volume"]].rename(columns={"src": "dst", "dst": "src", "volume": "rev_volume"})
    p = p.merge(rev, on=["src", "dst"], how="left")
    p["rev_volume"] = p["rev_volume"].fillna(0.0)
    p["s_rec"] = 1 - (p["volume"] - p["rev_volume"]).abs() / (p["volume"] + p["rev_volume"])
    # node flow balance: min(beta(a), beta(b))
    out_v, in_v = p.groupby("src")["volume"].sum(), p.groupby("dst")["volume"].sum()
    nodes = out_v.index.union(in_v.index)
    out_v, in_v = out_v.reindex(nodes, fill_value=0.0), in_v.reindex(nodes, fill_value=0.0)
    beta = 1 - (in_v - out_v).abs() / (in_v + out_v)
    p["s_bal"] = np.minimum(p["src"].map(beta), p["dst"].map(beta))
    # timing regularity: 1 / (1 + CV); fewer than 3 trades -> window median
    reg = 1 / (1 + p["gap_cv"])
    p["s_reg"] = reg.fillna(reg.median())
    return p

def rank_normalize(x):            # ties take the lowest rank, so the s_rec = 0 block maps to 0
    return (x.rank(method="min") - 1) / (len(x) - 1)
```

3.3 Persistence by boundary-matrix reduction (Section 2.3)

The pure-Python backend builds the directed flag complex up to dimension 2 and reduces the boundary matrix over Z/2. Each column is a set of row indices, and adding two columns is a symmetric difference.

```
for a in G.nodes():                       # 2-simplices (a,b,c): a->b, a->c, b->c
    for b in succ[a]:
        for c in succ[a] & succ[b]:
            cells.append((max(w[a, b], w[a, c], w[b, c]), 2, ("t", a, b, c)))
cells.sort(key=lambda c: (c[0], c[1]))   # filtration order

pivot, reduced = {}, []
```

```
for j, (_, dim, key) in enumerate(cells):
    col = boundary(key)                  # {} | {a, b} | {(b,c), (a,c), (a,b)} as row indices
    while col:
        low = max(col)
        if low in pivot:
            col ^= reduced[pivot[low]]   # column addition over Z/2
        else:
            pivot[low] = j               # (birth = cell low, death = cell j)
            break
    reduced.append(col)
```

3.4 Problem A: logistic regression by gradient descent (Section 2.5)

```
def logistic_loss_grad(theta, X, y, w, lam):
    p = sigmoid(X @ theta)
    reg = theta.copy(); reg[0] = 0.0                        # bias is not penalised
    J = -np.mean(w * (y * np.log(p) + (1 - y) * np.log(1 - p))) + 0.5 * lam * reg @ reg
    g = X.T @ (w * (p - y)) / len(y) + lam * reg            # (1/N) X^T Omega (p - y) + lambda theta~
    return J, g

def smoothness_constant(X, w, lam):                         # L = lambda_max(X^T Omega X)/(4N) + lambda
    M = (X * w[:, None]).T @ X / (4.0 * len(w))
    return np.linalg.eigvalsh(M).max() + lam

def gradient_descent(X, y, w, lam, eta, iters):
    theta = np.zeros(X.shape[1])
    for _ in range(iters):
        J, g = logistic_loss_grad(theta, X, y, w, lam)
        theta = theta - eta * g                             # theta_{t+1} = theta_t - eta grad J
    return theta
```

3.5 Problem B: projected gradient descent on the simplex (Section 2.6)

```
def project_simplex(u):                                     # sort-based KKT solution
    s = np.sort(u)[::-1]
    css = np.cumsum(s)
    j = np.arange(1, len(u) + 1)
    rho = np.nonzero(s - (css - 1) / j > 0)[0][-1] + 1
    nu = (css[rho - 1] - 1) / rho
    return np.maximum(u - nu, 0.0)

def loss_grad(self, alpha, c, k):
    w = 1.0 - self.Zpad @ alpha                             # w_e(alpha) for every edge of every ring
    bw = np.where(self.mask, self.beta * w, -np.inf)
    mx = bw.max(axis=1, keepdims=True)
    e = np.exp(bw - mx); S = e.sum(axis=1, keepdims=True)
    b = (mx[:, 0] + np.log(S[:, 0])) / self.beta            # smooth ring birth (log-sum-exp)
    pi = e / S                                              # softmax weights pi_{R,e}
    p = sigmoid(c - k * b)
    r = self.w * (p - self.y) / len(self.y)                 # residuals r_R
    zbar = np.einsum("nm,nmj->nj", pi, self.Zpad)           # sum_e pi_{R,e} z_e
    return bce(p), k * (r[:, None] * zbar).sum(0), r.sum(), -(r * b).sum()

for t in range(iters):                                      # projected gradient descent
    J, ga, gc, gk = learner.loss_grad(alpha, c, k)
    alpha = project_simplex(alpha - eta * ga)
    c, k = c - eta * gc, k - eta * gk
```

3.6 Problem C: online gradient ascent with delayed labels (Section 2.7)

```
def smoothed_reward_grad(tau, s, y, lam, T):
    g = y - lam * (1 - y)                                  # +1 for a catch, -lambda for a false alarm
    sg = sigmoid((tau - s) / T)
    return (g * sg * (1 - sg)).sum() / (T * len(s))

for t in range(n_windows):
    labelled = t - delay                                   # newest window whose labels are known
    if t >= warmup:
        s_l, y_l = windows[labelled]
        for _ in range(20):
            tau = np.clip(tau + eta * smoothed_reward_grad(tau, s_l, y_l, lam, T), 0, 1)   # ascent step
    rewards.append(reward(tau, *windows[t], lam))          # TP - lambda * FP on the new window
```

3.7 Verification

Check	Result
Logistic gradient vs central finite differences	max error < 1e-6
α, c, k gradients vs finite differences (Problem B)	max error 9.3e-11 on real training data
GD with η = 1/L decreases J every step (descent lemma)	holds on all 300 steps
GD solution vs SciPy L-BFGS on the same objective	max coefficient difference 6.5e-7
Simplex projection: worked example of Section 2.6, feasibility, optimality	(0.35, 0.65, 0); 200 random cases pass
Directed flag complex behaviour (Propositions 2–4) and pyflagser vs reference backend	all pass on 20 random graphs

To reproduce, run `conda activate ringtda`, then `pytest`, then `python experiments/gd_report.py` from the project folder. The run takes about 30 seconds and writes the five figures used in Section 4.

4. Results and Discussion

Gradient methods did three jobs. Learning the score weights by projected gradient descent raised the average precision of the ring-birth detector from 0.568 to 0.9997 on unseen windows. Gradient descent on the classifier converged to machine precision once the step size matched the local curvature. Under tactic drift, online gradient ascent on the threshold kept 97% of the best achievable reward, against 24% for a fixed threshold.

4.1 Experimental setup

Each window is one simulated week with 400 accounts and about 6,000 background transactions. Each window contains 9 planted wash rings (three each of 2, 3 and 5 accounts), 4 arbitrage loops, 2 market makers with 5 clients each, and 4 one-way decoy triangles. The wash rings' timing noise

(jitter) varies across windows from 5% to 60% of their trading period, so that no single score is sufficient.

Split	Windows	Timing jitter	Candidate rings	Wash rings	Market-maker loops	Arbitrage loops	Background loops
Train	12	0.05–0.60	2,235	108	120	47	1,960
Test	6	0.10–0.55	1,370	54	60	22	1,234

Test windows use different random seeds from training windows, so every test result is on data the models never saw. Every planted wash ring was recovered as a candidate in every window (candidate recall 1.0).

4.2 Problem A: gradient descent on the ring classifier

The classifier uses 13 features (8 behavioural, 5 topological) with $\lambda = 0.01$. The global smoothness constant from Section 2.5 is $L = 6.89$. The largest Hessian eigenvalue at the optimum is much smaller, $L_{local} = 0.221$.

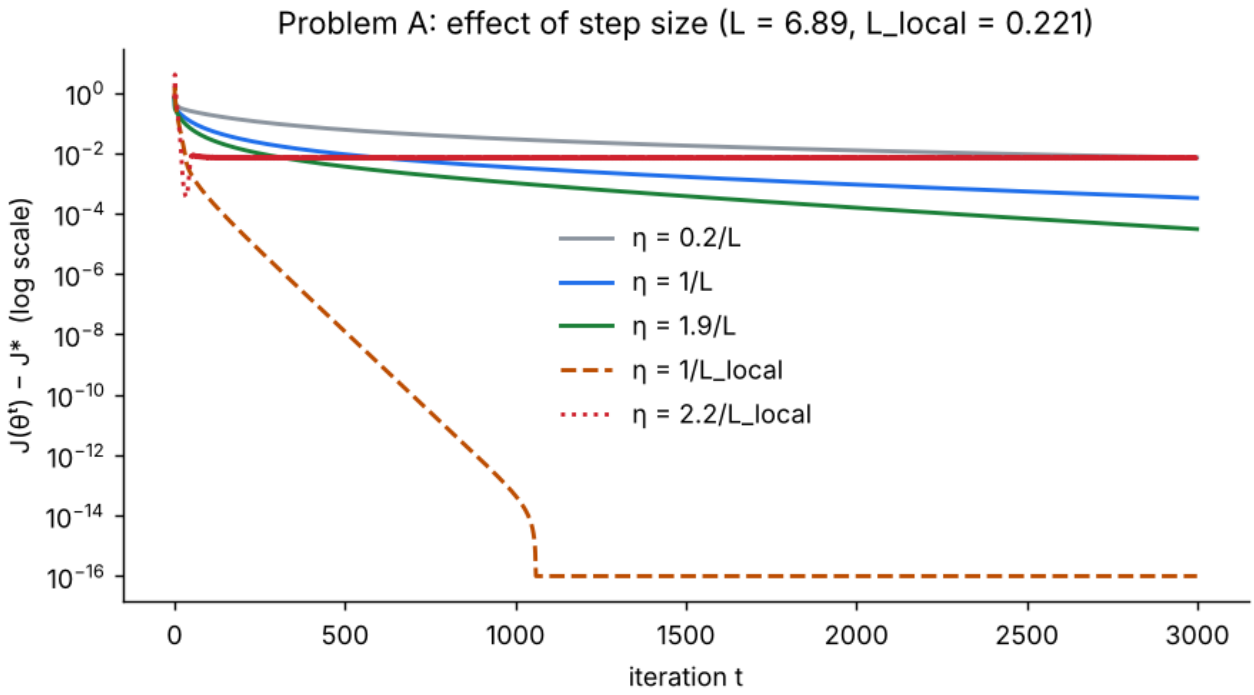


Figure 1: Optimality gap $J(\theta^t) - J^*$ on a log scale for five step sizes; J^* from SciPy L-BFGS.

Step size η	Gap after 3,000 iterations	Iterations to reach a gap of 1e-6
0.2/L	7.1e-3	not reached
1/L (textbook safe step)	3.3e-4	not reached

Step size η	Gap after 3,000 iterations	Iterations to reach a gap of 1e-6
1.9/L	3.1e-5	not reached
1/L_local	about 1e-16 (machine precision)	322
2.2/L_local	7.4e-3 (stalls, oscillates)	not reached

Interpretation.

- The bound L assumes the worst case p(1 − p) = 1/4 for every ring, as if every prediction were a coin flip. Near the optimum most rings are classified confidently, so p(1 − p) is close to 0 and the true curvature is 31 times smaller. The "safe" step 1/L is therefore very conservative here.
- The step-size theory is confirmed at the local scale. η = 1/L_local converges linearly (a straight line on the log plot), as predicted by the strong-convexity rate. η = 2.2/L_local, just past the 2/L_local stability limit, stops making progress and oscillates around the optimum.
- The gradient-descent solution matches SciPy's L-BFGS to within 6.5e-7 in every coefficient, which confirms the gradient derivation.

Detection quality. On the test windows, the classifier reaches AP = 0.998 with behavioural features alone and 0.997 with behavioural plus topological features. Topology adds no measurable lift on this data. This matches the expectation recorded in the project proposal: persistence adds information mainly when neighbouring trades fill or split a loop (Proposition 4), and these synthetic rings are already separated by their behaviour. It is reported here as a finding rather than tuned away.

The largest standardized coefficients are min_reg (+1.08, regular timing on every edge), neighbourhood size (−0.80), mean_reg (+0.54) and mean_rec (−0.39). Reciprocity has a *negative* weight: on this data, two-way trading is more typical of market makers than of wash rings.

4.3 Problem B: learning the score weights on the simplex

Projected gradient descent (η = 0.5, β = 30, 1,500 iterations) reduced the weighted cross-entropy from 0.883 to 0.0794 and converged to:

$$\alpha^* = (\alpha_{\text{rec}},\, \alpha_{\text{bal}},\, \alpha_{\text{reg}}) = (0.000,\ 0.207,\ 0.793)$$

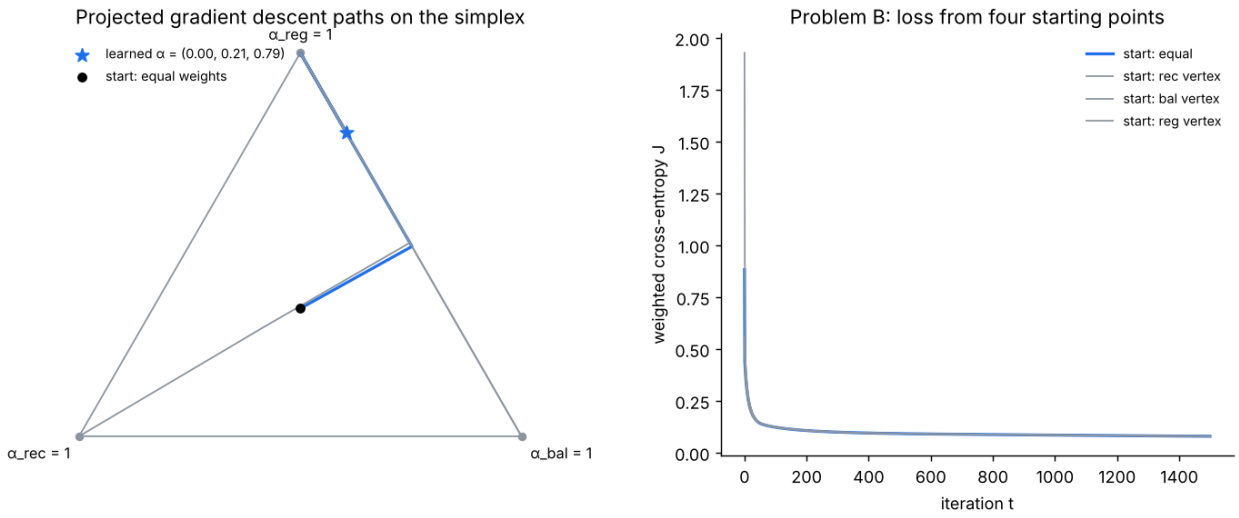


Figure 2: Left: PGD paths on the simplex from four starting points (equal weights and the three corners), all ending at the same point. Right: loss curves.

Starting from equal weights and from each of the three corners, PGD ended at the same point (α_bal between 0.205 and 0.207; final losses within 0.0002). The problem is non-convex, so this is not a proof of global optimality, but there is no sign of competing local minima.

KKT check at the solution. The gradient of J with respect to α at α* is (0.0848, −0.0511, −0.0505). The two weights in use (balance and regularity) have equal partial derivatives, so $\nu \approx 0.051$. The dropped weight, reciprocity, has a partial derivative of +0.0848, which is at least $-\nu$. These are exactly the stationarity conditions of Section 2.6: moving any weight onto reciprocity would increase the loss.

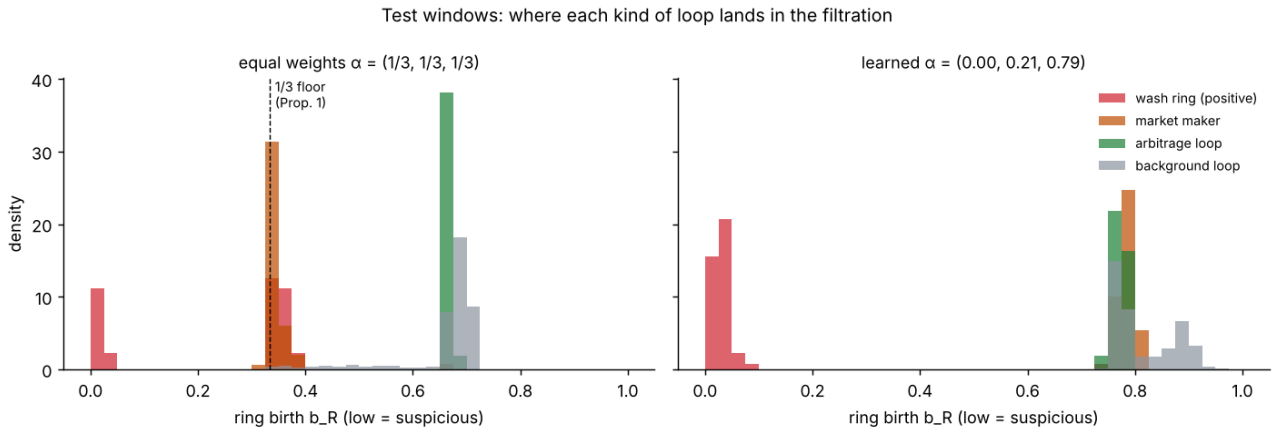


Figure 3: Ring birth on the test windows by kind of loop. Left: equal weights; market makers pile up on the 1/3 floor next to the multi-party wash rings. Right: learned weights; all wash rings fall below 0.1 and all legitimate loops above 0.72.

Why the learned weights work. Figure 3 (left) shows Proposition 1 in the data. With equal weights, multi-party wash rings cannot score below 1/3, and market makers, which are reciprocal and roughly balanced, land on the same value. The detector cannot separate them. Gradient

descent fixes this by moving all weight off reciprocity (it describes market makers as well as wash rings) and onto timing regularity, which only bots on a schedule show.

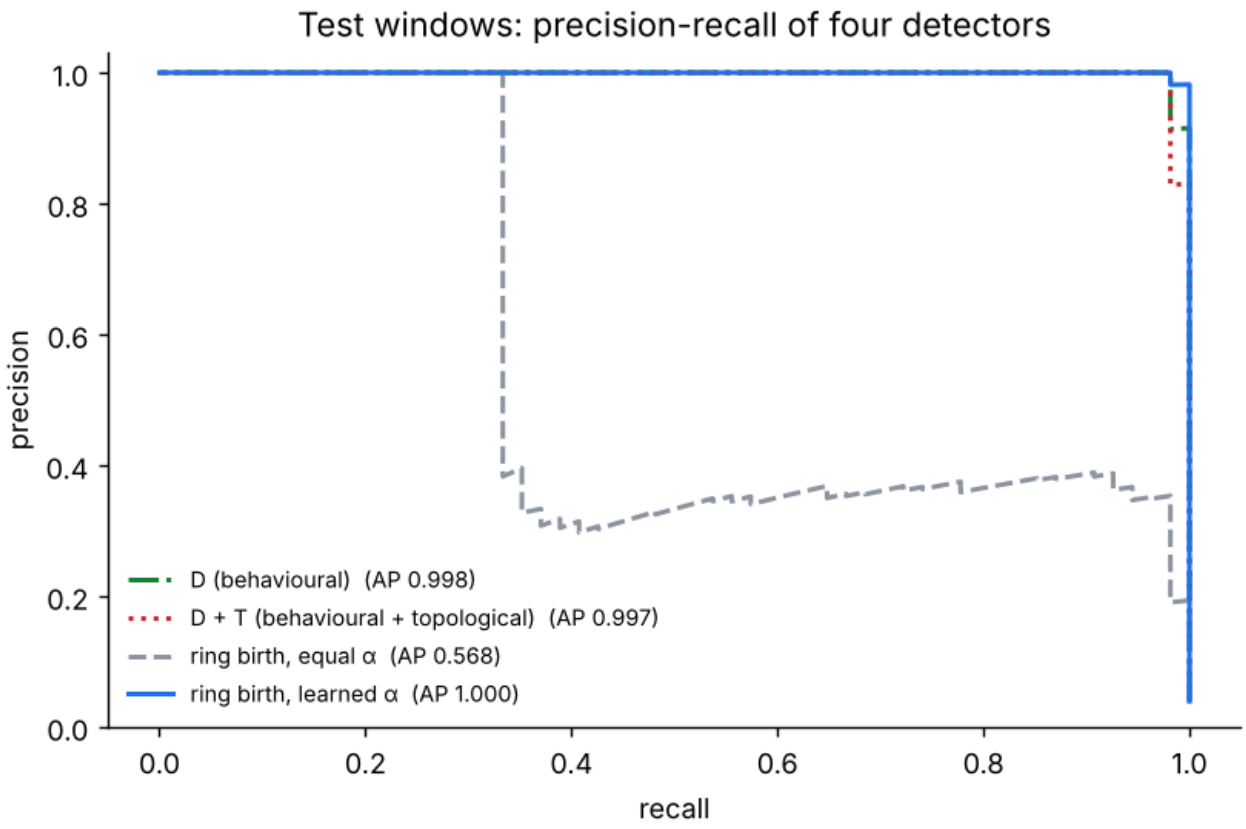


Figure 4: Precision–recall curves on the test windows for four detectors.

Using ring birth alone as the score, test AP rises from 0.568 (equal weights) to 0.9997 (learned weights). A single learned filtration then matches the 13-feature classifier.

4.4 Problem C: adaptive threshold under drift

The deployed detector was learned only on four early, low-noise windows. Its weights went to the regularity corner, $\alpha = (0, 0, 1)$. Over the next 40 windows the manipulators adapt: timing jitter rises from 0.05 to 0.60 and amount noise from 0.01 to 0.16. Labels arrive 2 windows late; $\lambda = 1$, $T = 0.02$, $\eta = 0.05$, with 20 ascent steps per window.

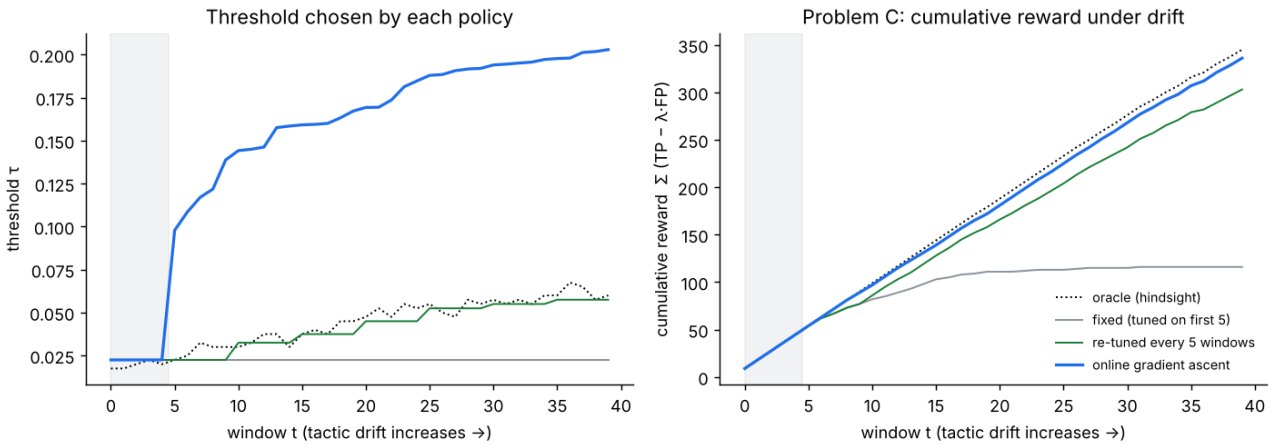


Figure 5: Left: threshold chosen by each policy over time; grey band = warm-up. Right: cumulative reward TP − λ·FP.

Policy	Cumulative reward after warm-up	Share of oracle
Hindsight oracle (upper bound)	300	100%
Online gradient ascent	291	97%
Re-tuned by grid search every 5 windows	258	86%
Fixed threshold (tuned on the first 5 windows)	71	24%

Interpretation.

- The fixed threshold stops catching rings once their timing noise pushes their birth values above it, and its reward flattens out by window 15.
- Gradient ascent does better than periodic grid search, although both see the same delayed labels. The reward is flat across the gap between wash rings and legitimate loops, so any threshold in the gap scores the same today. Grid search returns the *smallest* such threshold, which sits right against the current wash rings and misses rings that drift upward before the next re-tune. The smoothed objective balances the sigmoid tails of the nearest positives and negatives, which places τ near the *middle* of the gap. That is a margin-like choice that keeps catching rings as they drift.
- **Control experiment.** With the robust weights from Problem B, which were trained on the whole range of timing noise, all three policies reach 94–95% of the oracle (fixed 292, re-tuned 287, ascent 291, oracle 306). Threshold adaptation matters for drift the training data did not cover. Learning on diverse tactics and adapting online are complementary defences.

4.5 How gradient descent and ascent help with this problem

1. **They turn design choices into objectives.** The equal 1/3 weighting looked neutral but was the main reason the baseline failed (Proposition 1). Writing the weighting as a parameter and minimising a loss let the data choose it, raising AP from 0.568 to 0.9997.
2. **Smoothing makes the problem differentiable.** Ring birth uses a max and flagging uses an indicator, so both have zero or undefined gradients. Log-sum-exp and sigmoid surrogates give useful gradients, and the softmax weights have a direct meaning: the update is driven by each ring's weakest link.

3. **Constraints keep the answer interpretable.** Projecting onto the simplex keeps α as shares of evidence, and the KKT conditions turn the result into a statement: reciprocity is uninformative here (weight exactly 0), and balance and regularity contribute equally at the margin.
4. **Convergence theory guides practice.** The descent lemma explains why 1/L is safe and why it was slow. Measuring the local curvature gave a step 31× larger and convergence in 322 iterations.
5. **Online ascent handles drift with delayed feedback.** Each new labelled window gives a cheap gradient step, so the threshold tracks moving tactics without a full re-optimisation, and its margin-like solution is more robust than grid search.

4.6 Limitations

- All results are on synthetic data with known ground truth. Real proxy labels (IBM AML, NFT wash-trade flags) will be noisier, and the near-perfect APs here should not be expected there.
- Topological features gave no lift on this data. Scenarios where neighbours fill or split loops (Proposition 4) still need to be generated and tested.
- Problem B is non-convex. Agreement across four starts is evidence, not proof, of a global minimum.
- The smoothing temperatures (β, T) trade bias against gradient signal. When no candidate lies within a few T of τ, the ascent gradient vanishes and the threshold stops moving.
- The threshold study assumes delayed labels are correct. In practice confirmations are incomplete and biased toward flagged cases.

5. Conclusion and Future Work

Detecting wash-trading rings can be posed as a set of optimization problems, and gradient methods solve each one with a clear mathematical guarantee or interpretation. The directed flag complex identifies loops that represent circulating value (Propositions 2–4). Gradient descent then learns how to weight the evidence on each edge, how to classify candidate rings, and where to set the alarm threshold as tactics change.

The main findings:

- The equal-weight filtration has a structural flaw: no multi-party ring can score below 1/3 (Proposition 1), which is exactly where market makers sit. Projected gradient descent on the simplex removes it by learning $\alpha = (0, 0.21, 0.79)$, lifting test AP from 0.568 to 0.9997, and the KKT conditions confirm the solution.
- For the convex classifier, the step size set by local curvature (1/L_local) converged in 322 iterations, while the global bound 1/L had not converged after 3,000. Steps beyond 2/L_local oscillated, as theory predicts.
- Under tactic drift, online gradient ascent on a smoothed reward retained 97% of the oracle reward, against 86% for periodic re-tuning and 24% for a fixed threshold. A detector trained on diverse tactics needed almost no adaptation.

Future work.

1. Run the same pipeline on the IBM AML benchmark and Ethereum NFT trades, where labels are noisier and the classes overlap more.
2. Generate the scenarios where topology should matter (dense neighbourhoods, chorded rings, loops filled through other accounts) and test whether the topological features add lift

there.

3. Replace full-batch gradient descent with stochastic (mini-batch) gradient descent for windows with millions of transactions.
4. Model delayed and incomplete feedback in Problem C more realistically, for example labels only for flagged rings.

References

Listed from memory; check the details before submission.

1. S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
2. Y. Nesterov. *Introductory Lectures on Convex Optimization: A Basic Course*. Kluwer, 2004.
3. J. Duchi, S. Shalev-Shwartz, Y. Singer and T. Chandra. Efficient projections onto the l1-ball for learning in high dimensions. *ICML*, 2008.
4. M. Zinkevich. Online convex programming and generalized infinitesimal gradient ascent. *ICML*, 2003.
5. H. Edelsbrunner and J. Harer. *Computational Topology: An Introduction*. American Mathematical Society, 2010.
6. A. Zomorodian and G. Carlsson. Computing persistent homology. *Discrete & Computational Geometry* 33, 2005.
7. D. Lütgehetmann, D. Govc, J. P. Smith and R. Levi. Computing persistent homology of directed flag complexes. *Algorithms* 13(1), 2020.
8. E. Altman et al. Realistic synthetic financial transactions for anti-money laundering models. *NeurIPS Datasets and Benchmarks*, 2023.
9. L. W. Cong, X. Li, K. Tang and Y. Yang. Crypto wash trading. *Management Science*, 2023.
10. V. von Wachter, J. R. Jensen, F. Regner and O. Ross. NFT wash trading: quantifying suspicious behaviour in NFT markets. 2022.

Appendix A: Derivations to work out on the board (viva)

A1. Derivative of the sigmoid.

$$\sigma(u) = (1+e^{-u})^{-1} \;\Rightarrow\; \sigma'(u) = \frac{e^{-u}}{(1+e^{-u})^2} = \frac{1}{1+e^{-u}} \cdot \frac{e^{-u}}{1+e^{-u}} = \sigma(u)\,(1-\sigma(u))$$

A2. Gradient of the cross-entropy. With $p = \sigma(u)$, $u = \theta^{\top}x$ and loss $\ell = -y\log p - (1-y)\log(1-p)$:

$$\frac{\partial \ell}{\partial u} = -\frac{y}{p}\,p(1-p) + \frac{1-y}{1-p}\,p(1-p) = -y(1-p) + (1-y)p = p - y, \qquad \nabla_{\theta}\ell = (p-y)\,x$$

A3. Convexity. For any direction v, the Hessian quadratic form is a sum of non-negative terms:

$$v^{\top}\nabla^2 J\, v = \frac{1}{N}\sum_{R} \omega_R\, p_R(1-p_R)\,(x_R^{\top}v)^2 + \lambda\|\tilde{v}\|^2 \;\geq\; 0$$

A4. Descent lemma. L-smoothness gives the quadratic upper bound; substituting the gradient

step gives the decrease:

$$J(\theta') \le J(\theta) + \nabla J(\theta)^\top(\theta' - \theta) + \frac{L}{2}\|\theta' - \theta\|^2$$

With the gradient step θ′ = θ − η∇J(θ):

$$J(\theta') \le J(\theta) - \eta\Big(1 - \frac{L\eta}{2}\Big)\|\nabla J(\theta)\|^2$$

A5. Why the step must be below 2/L. On the one-dimensional quadratic J(θ) = Lθ²/2:

$$\theta_{t+1} = \theta_t - \eta L\theta_t = (1 - \eta L)\,\theta_t \;\Rightarrow\; \theta_t = (1 - \eta L)^t\theta_0 \to 0 \iff |1 - \eta L| < 1 \iff 0 < \eta < \frac{2}{L}$$

At η = 1/L it converges in one step; between 1/L and 2/L it overshoots but still converges; above 2/L it diverges. Figure 1 shows this at the local curvature L_local.

A6. Bounds on the soft maximum. With m = max over the ring's edges of w_e:

$$e^{\beta m} \le \sum_{e\in R} e^{\beta w_e} \le |R|\, e^{\beta m} \;\Rightarrow\; m \le \frac{1}{\beta}\log\sum_{e\in R} e^{\beta w_e} \le m + \frac{\log|R|}{\beta}$$

A7. Gradient of the soft maximum.

$$\frac{\partial}{\partial w_e}\frac{1}{\beta}\log\sum_{e'} e^{\beta w_{e'}} = \frac{1}{\beta}\cdot\frac{\beta\, e^{\beta w_e}}{\sum_{e'} e^{\beta w_{e'}}} = \pi_{R,e}, \qquad \frac{\partial w_e}{\partial \alpha} = -z_e$$

A8. Projection onto the simplex. Lagrangian, KKT conditions, the max(u − ν, 0) solution, and the worked example u = (0.5, 0.8, −0.1) giving (0.35, 0.65, 0), all as in Section 2.6. Check: 0.35 + 0.65 + 0 = 1, and the components that stay positive are each shifted down by the same ν = 0.15.

A9. Smoothed threshold gradient and its limit.

$$\frac{d}{d\tau}\,\sigma\Big(\frac{\tau - s}{T}\Big) = \frac{1}{T}\,\sigma(1-\sigma), \qquad \int_{-\infty}^{\infty}\frac{1}{T}\,\sigma\Big(\frac{x}{T}\Big)\Big(1 - \sigma\Big(\frac{x}{T}\Big)\Big)\,dx = 1$$

The factor integrates to 1 and narrows as T → 0, so it tends to a delta function. The gradient then counts the density of positives minus λ times the density of negatives at τ, which gives the optimality condition of Section 2.7.

A10. Why a cyclic triangle is never filled (Proposition 2). A directed 2-simplex (v_0, v_1, v_2) needs v_0 to have edges to both other vertices. In A → B → C → A each vertex has exactly one outgoing edge inside the triangle, so no vertex can be v_0 and no 2-simplex exists. The cycle has zero boundary, (B + A) + (C + B) + (A + C) = 0 over Z/2, but is not a boundary, so it is a non-trivial H_1 class that never dies.

Likely viva questions, with one-line answers.

Question	Answer
Why gradient descent and not a closed-form solution?	Logistic regression has no closed form; the gradient equation $p(\theta) = y$ is non-linear in θ.
Why is Problem A convex but Problem B not?	A is a convex loss of a linear function of θ; B composes a sigmoid with a soft maximum of α, which breaks convexity.
Why project instead of using a penalty for the constraint?	Projection keeps every iterate exactly feasible (true shares), and the simplex projection costs only a sort.
What does $\alpha_{rec} = 0$ mean?	The KKT conditions say adding weight to reciprocity would increase the loss: two-way trading also describes market makers, so it misleads.
Why smooth the max and the indicator?	Their gradients are zero almost everywhere (indicator) or undefined at ties (max), so gradient methods cannot move.
Why did 1/L converge slowly?	L uses the worst case $p(1 - p) = 1/4$; near the optimum the true curvature was 31 times smaller.
Why gradient ascent and not descent in Problem C?	The objective is a reward (catches minus cost of false alarms) to be maximised; ascent is descent on its negative.